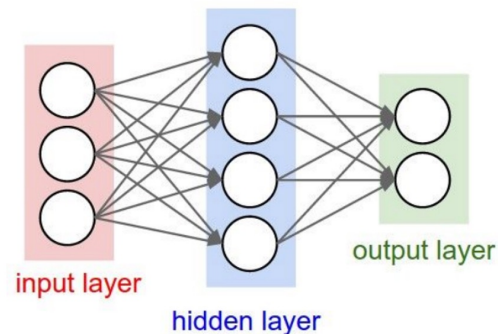


Building Our First *Neural* LM

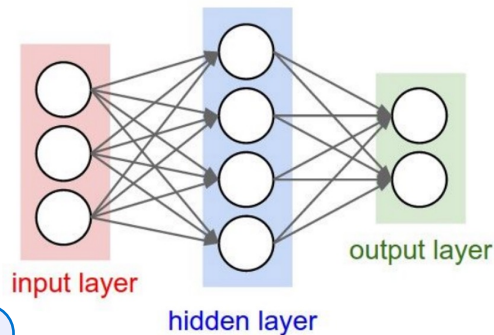
CSCI 601-471/671 (NLP: Self-Supervised Models)

Recap: Neural Nets

- A powerful function-approximation tool.
- Can be trained efficiently via Backpropagation.
- Out focus here: how to use NNs for language modeling.



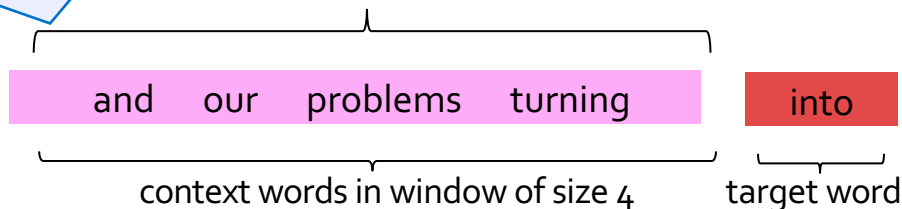
Big Picture: Language Modeling + NNs



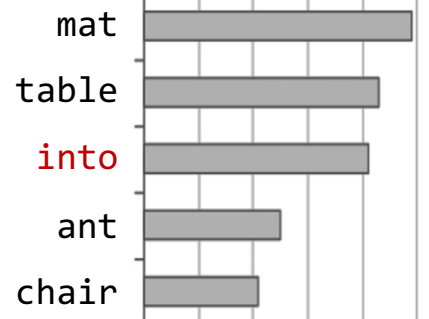
How do MLPs are for Language modeling?

Remember NNs expect numbers. How do you feed a string to neural net?

$$f(\mathbf{x}, \Theta)$$



Probs over vocabulary



Building First Neural LMs

1. Feeding text to NNs
2. Fixed-window neural language models

Chapter goal: Get more comfortable with thinking about the role of neural networks in modeling distribution of language.

Feeding Text to Neural LMs

Feeding Text to Neural Nets

- Neural Nets expect numbers.
- How do you turn numbers into numbers?

Feeding Text to Neural Nets

Text Input

The movie was great!



Tokenized Input

The movie was great!



Embedded Input

$$X = [x_1 \ x_2 \ x_3 \ x_4 \ x_5] \in \mathbb{R}^{d \times 5}$$



- How do you actually look up these embeddings?
- How can you train these embeddings?
- Idea:
 - (1) maintain an embedding matrix for all the words.
 - (2) this embedding matrix can be counted as a **trainable** layer in your neural net.

Embedding lookup: example

I love Peperroni Pizza



tokenization

Embedding lookup: example

I love Peperoni Pizza



tokenization



["I ", " _love ", " _Pep", "per", "oni", " _pizza"]

Embedding lookup: example

I love Peperroni Pizza



tokenization

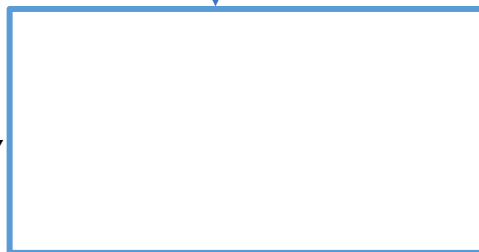


["I ", " _love ", " _Pep", "per", "oni", " _pizza"]



Embedding
matrix

d



Vocab size

Embedding lookup: example

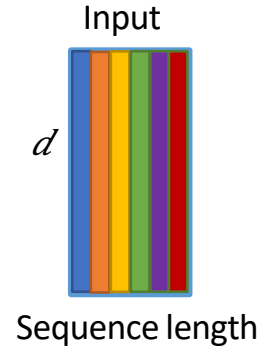
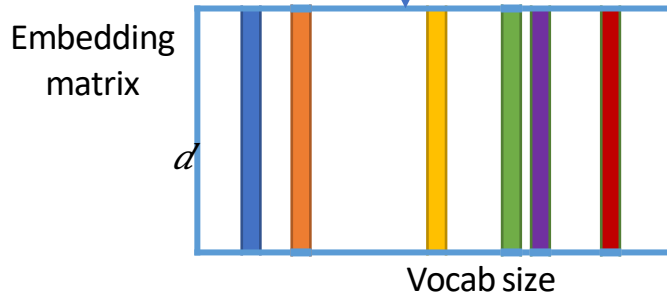
I love Pepperroni Pizza



tokenization



["I ", "_love ", "_Pep", "per", "oni", "_pizza"]

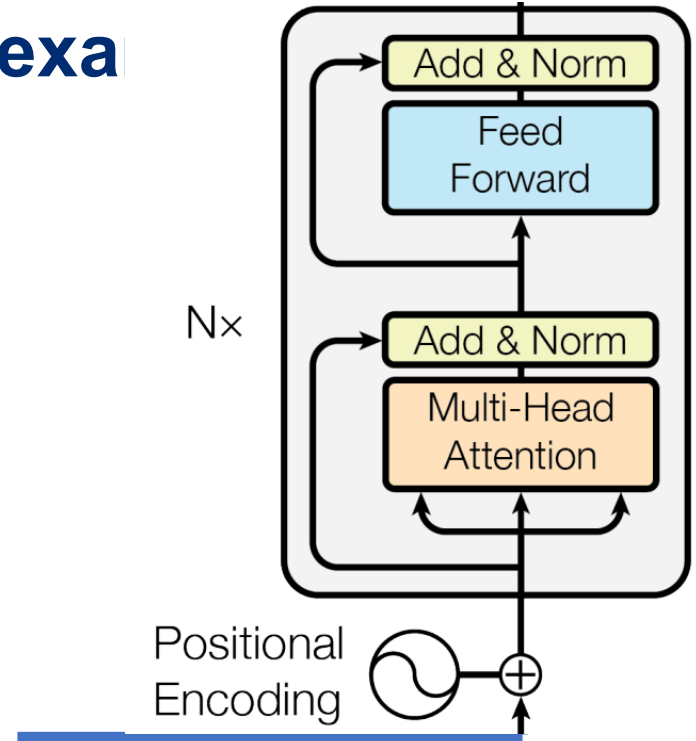
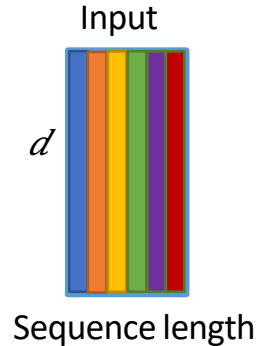
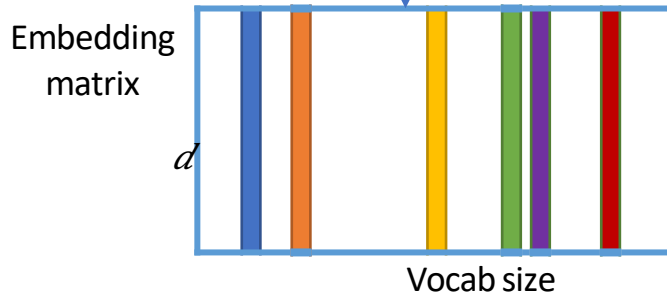


Embedding lookup: example

I love Pepperroni Pizza

tokenization

["I ", " _love ", " _Pep", "per", "oni", " _pizza"]



The input encoding pipeline: In Practice

Initialize a random embedding matrix

Indices corresponding to input units (tokens)

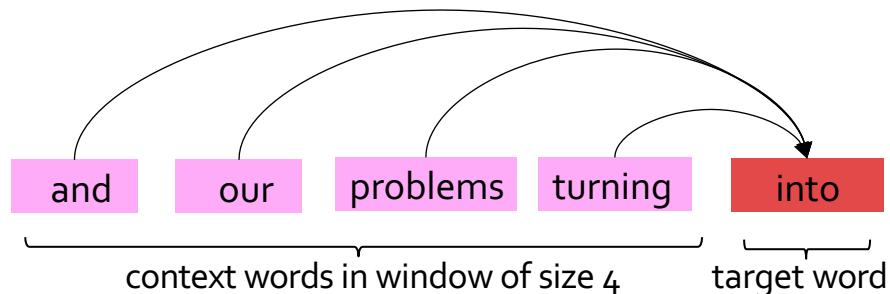
Retrieved the embeddings corresponding to the token ids with **index lookup**

```
# an Embedding module containing 10 tensors of size 3
n, d = 10, 3
embedding = nn.Embedding(n, d)
# a batch of 2 samples of 4 indices each
input = torch.LongTensor([[1, 2, 4, 5], [4, 3, 2, 9]])
embedding(input)
tensor([[[[-0.0251, -1.6902,  0.7172],
          [-0.6431,  0.0748,  0.6969],
          [ 1.4970,  1.3448, -0.9685],
          [-0.3677, -2.7265, -0.1685]],
         [[ 1.4970,  1.3448, -0.9685],
          [ 0.4362, -0.4004,  0.9400],
          [-0.6431,  0.0748,  0.6969],
```

Fixed-Window MLP Language Models

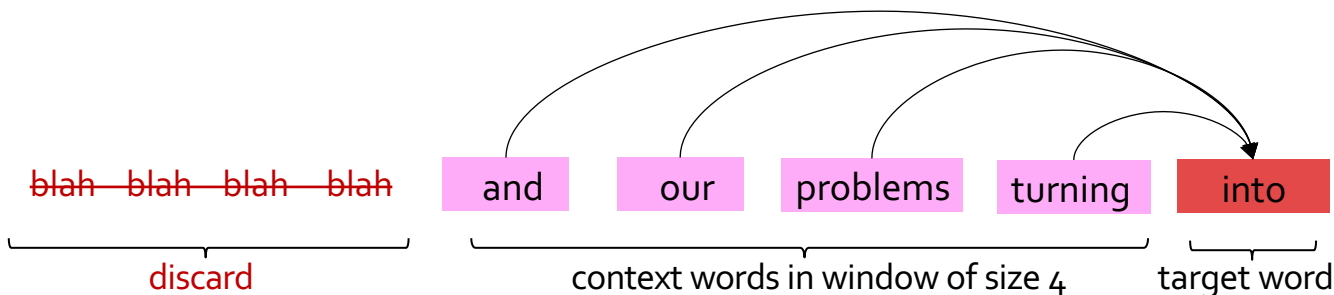
A Fixed-Window Neural LM

- Given the embeddings of the **context**, predict a **target word** on the right side.
 - Dropping the right context for simplicity -- not a fundamental limitation.



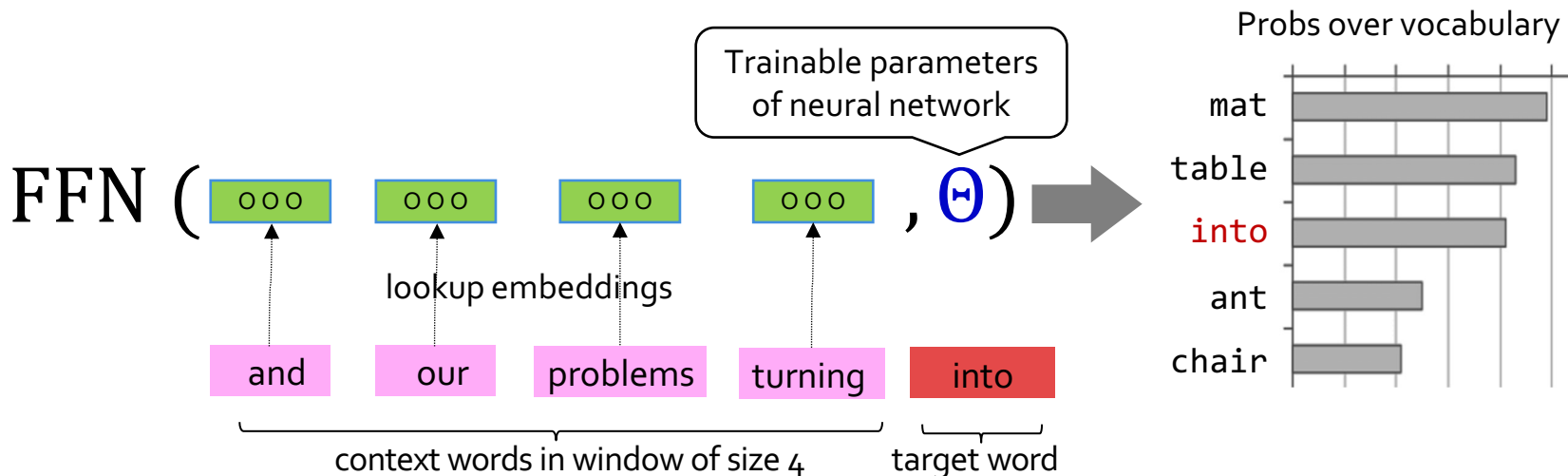
A Fixed-Window Neural LM

- Given the embeddings of the **context**, predict a **target word** on the right side.
 - Dropping the right context for simplicity -- not a fundamental limitation.
- Discard anything beyond its context window



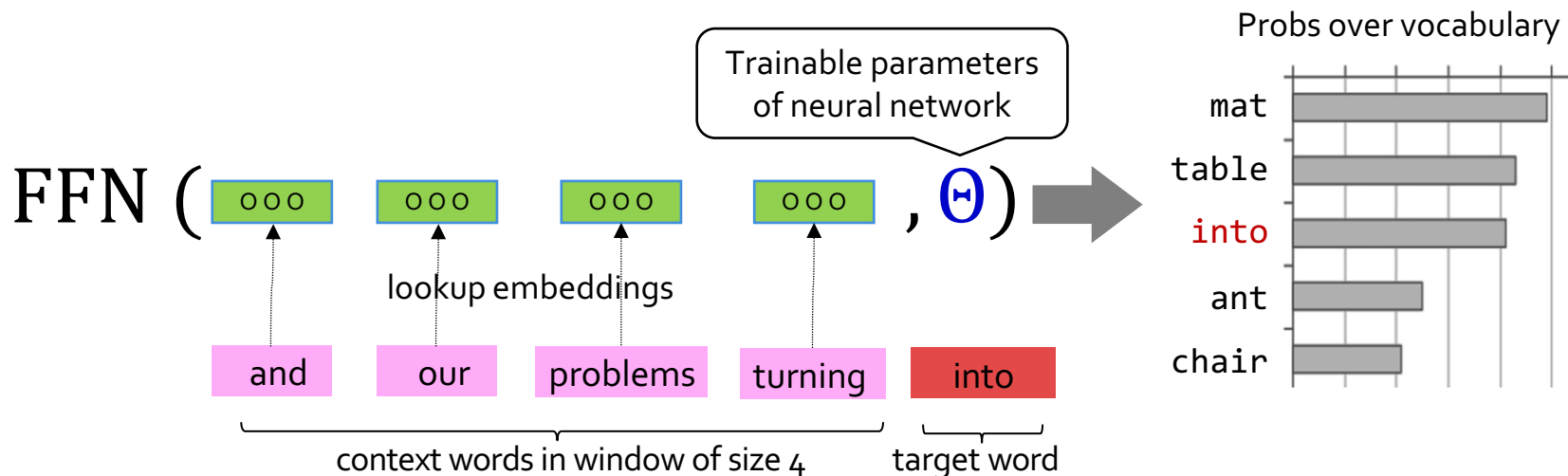
A Fixed-Window Neural LM

- Given the embeddings of the **context**, predict a **target word** on the right side.
 - Dropping the right context for simplicity -- not a fundamental limitation.
- Training this model is basically optimizing its parameters θ such that it assigns high probability to the **target word**.

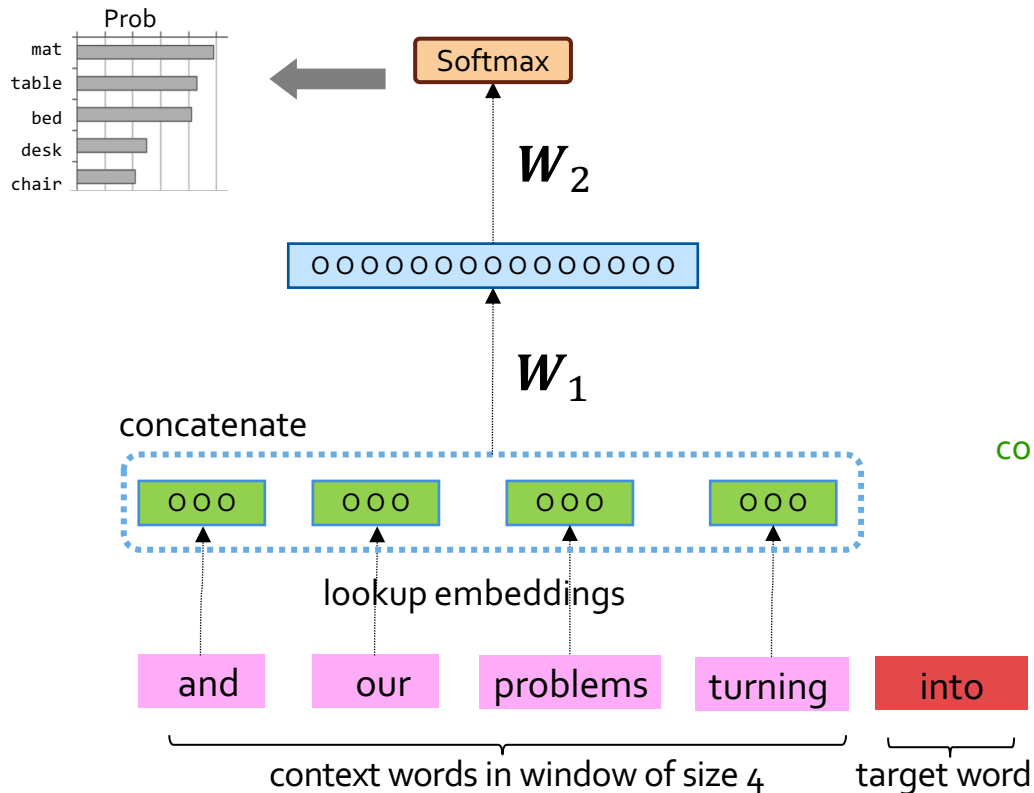


A Fixed-Window Neural LM

- This is actually a pretty good model!
- It will also lay the foundation for the future models (e.g., transformers, ...)
- But first we need to figure out how to train neural networks!



A Fixed-Window Neural LM



output distribution

$$y = \text{softmax}(W_2 h)$$

hidden layer

$$h = f(W_1 x)$$

concatenated word embeddings

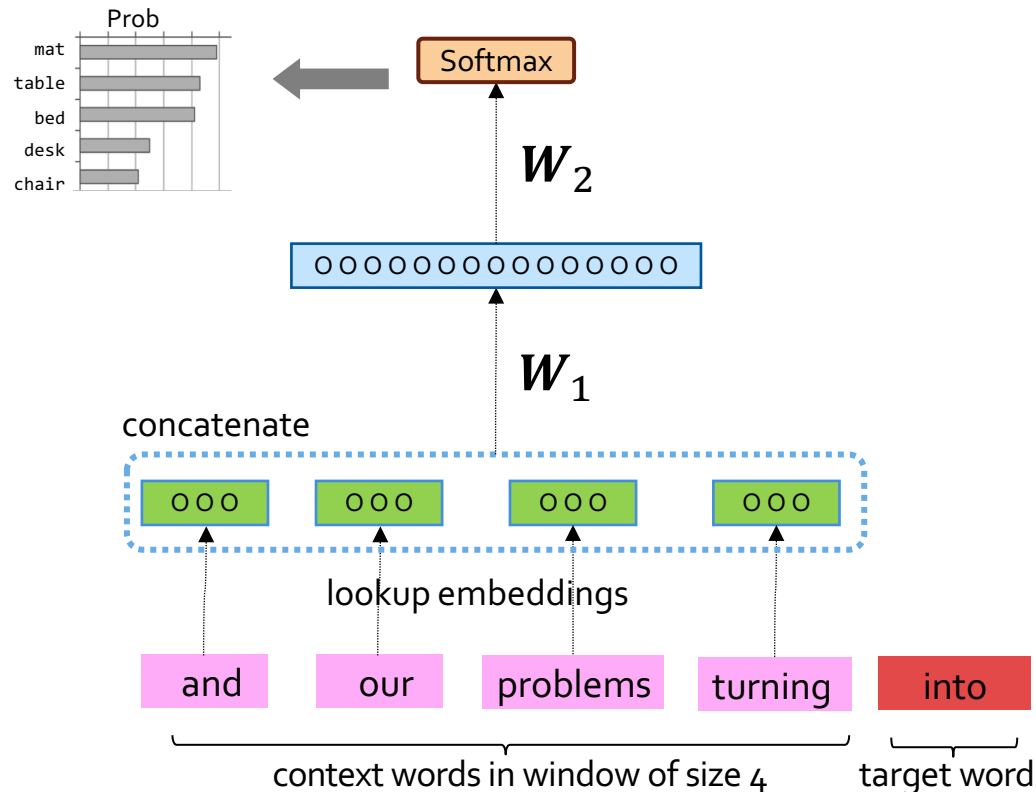
$$x = [v_1, v_2, v_3, v_4]$$

A Fixed-Window Neural LM: Compared to N-Grams

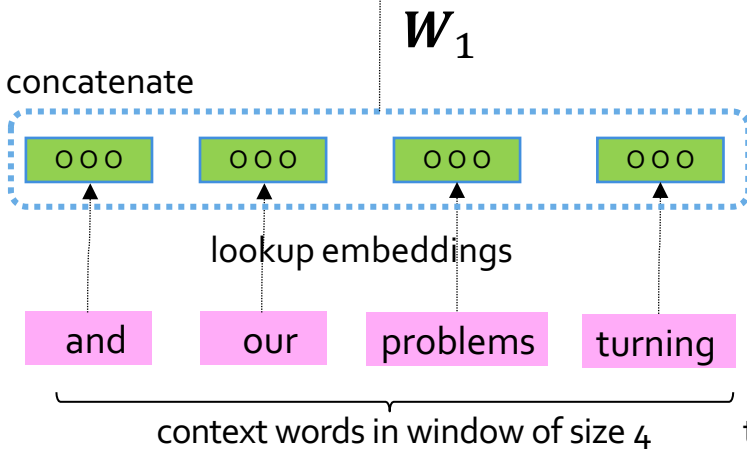
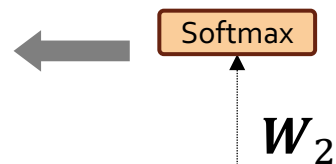
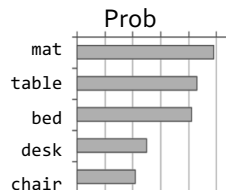
Improvements over n-gram LM:

- Tackles the sparsity problem
- Model size is $O(n)$ not $O(\exp(n))$ — n being the window size.

	n	valid.	test.
MLP10	6	104	109
Back-off KN	3	121	127
Back-off KN	4	113	119
Back-off KN	5	112	117



A Fixed-Window Neural LM: Compared to N-Grams



Remaining **problems**:

- Fixed window is too small
- Enlarging window enlarges W — Window can never be large enough!
- It's not deep enough to capture nuanced contextual meanings

What Changed from N-Gram LMs to Neural LMs?

- What is the source of Neural LM's **strength**?
- Why **sparsity** is less of an issue for Neural LMs?
- **Answer:** In n-grams, we treat all prefixes independently of each other! (even those that are semantically similar)

students opened their ____
pupils opened their ____
scholars opened their ____
undergraduates opened their ____
students turned the pages of their ____
students attentively perused their ____
...

Neural LMs are able to share information across these semantically-similar prefixes and overcome the sparsity issue.

Summary

- **Language Modeling (LM)**, a probabilistic model of language
- **N-gram models** (~1980 to early 2000's)
 - Difficult to scale to large n's
- **Fixed-window Neural LM**: first of many LMs we will see in this class
 - Stronger than n-gram LMs
 - But still fail at capturing longer contexts
- **Next**: other architectural alternatives: RNNs.